

Cherry River — Project Handoff Document

Date: March 13, 2026 **Prepared by:** Ahmed **Prepared for:** Francis **Supabase project:** Cherry River Core Data Supabase **Retool workspace:** cherryriver2.retool.com

What This Project Does

Automated weekly data pipeline for Cherry River's SAQ (Quebec liquor board) sales data. A Selenium bot logs into the SAQ B2B portal every Monday, downloads 6 CSV/ZIP reports, processes them, and uploads to Supabase. A Retool dashboard then reads those tables through SQL views.

There is also a secondary project in progress: the **Sales Rep Cockpit** — a Retool table replicating the WordPress SAQ sales table at saq.cherryriver.ca.

Repository Structure

```
CherryRiver/
├── run_weekly_pipeline.py      ← MAIN ORCHESTRATOR (runs all 3 steps)
├── saq_data_scraper.py        ← Selenium: logs into SAQ portal, downloads
├── CSVs
│   ├── requirements.txt       ← Python dependencies
│   └── .env.example           ← Copy to .env and fill in secrets
├── scripts/
│   ├── saq_weekly_update.py   ← CSV → Supabase uploader
│   ├── unzip_saq_files.py     ← Extracts ZIPs, deletes them
│   ├── create_dashboard_views.sql ← ALL Supabase views (deploy here)
│   ├── create_daily_tables.sql ← Schema for daily snapshot tables
│   └── create_saq_lead_times.sql ← Lead times migration
├── salesrep_cockpit/
│   ├── HANDOFF.md             ← NEW sub-project (Sales Rep Cockpit)
│   └── sql/
│       └── vw_cockpit_product_summary.sql ← DETAILED HANDOFF FOR THIS SUB-PROJECT
├── .github/
│   └── workflows/
│       └── saq_weekly_update.yml ← GitHub Actions: runs every Monday 8 AM UTC
├── n8n_workflows/             ← Patrick's daily automation exports
├── mappings/
│   └── mapping)               ← CSV reference data (territories, product
├── Retool Dashboards/         ← JSON exports of Retool dashboards
└── Project_Info/              ← Meeting notes, transcripts, PDFs
```

Environment Variables

Copy `.env.example` to `.env` and fill in:

```
SAQ_USERNAME=<saq b2b login>
SAQ_PASSWORD=<saq b2b password>
SUPABASE_URL=https://nqxqqoinpoomcqdddoqq.supabase.co
SUPABASE_KEY=<supabase service role key>
```

The same 4 variables must be added as **repository secrets** in GitHub (Settings → Secrets → Actions) for the automated workflow to work.

Running the Pipeline Manually

```
# Full pipeline
python run_weekly_pipeline.py

# Individual steps
python saq_data_scraper.py
python scripts/unzip_saq_files.py
python scripts/saq_weekly_update.py --folder "SAQ Documents 2"
```

Automated Weekly Pipeline (GitHub Actions)

File: `.github/workflows/saq_weekly_update.yml` **Schedule:** Every Monday at 8:00 AM UTC (3:00 AM EST)
Can also be triggered manually from the GitHub Actions tab (workflow_dispatch).

What it does:

1. Checks out the repo on an Ubuntu runner
2. Installs Python 3.11 + pip dependencies
3. Installs Chrome + ChromeDriver via `browser-actions/setup-chrome`
4. Installs Xvfb (virtual display — Chrome runs visibly on a virtual screen, NOT headless)
5. Runs `python run_weekly_pipeline.py` with `DISPLAY=:99`
6. Uploads the downloaded CSV files as artifacts (kept 7 days, useful for debugging)

Important: Uses Xvfb (not headless mode) because the SAQ portal has anti-bot measures that block headless Chrome.

Supabase Tables

Weekly Tables (uploaded by pipeline every Monday)

Table	What it contains	Upload mode
-------	------------------	-------------

Table	What it contains	Upload mode
saq_ventes	Store-level sales, bottles per period/week	APPEND only — never deleted
saq_inventaire	Store inventory snapshots	UPSERT weekly
saq_inventaire_entrepot	Warehouse inventory (legacy)	UPSERT weekly
saq_commandes	Purchase orders	APPEND, deduplicated by no_commande
saq_product	~80 Cherry River products (static)	Manual

Daily Snapshot Tables (Patrick's n8n automation)

Table	What it contains	Updated
saq_daily_warehouse	Warehouse stock CDM/CDQ in cases + uvc + inv_alloc_cdm	Daily
saq_daily_stores	Store-level bottle inventory	Daily
saq_order_status	Open SAQ purchase orders	Daily

⚠ **Recurring maintenance:** The Google Drive credentials used by the daily n8n workflow expire every 6 days and must be refreshed manually. Patrick handles this. If saq_daily_warehouse stops updating, check the n8n workflow — Google Drive re-auth is the most likely cause.

Reference Tables

Table	What it contains
saq_product	Product catalog — format_caisse (bottles/case), product_type, commission_rate
lead_times	Lead time per product in days — real values already populated
product_saq_mapping	Links Odoo product IDs → SAQ codes

Unit Conventions

Critical — mixing units causes silent errors:

Column	Unit	To convert
saq_ventes.bouteille	bottles	÷ saq_product.format_caisse → cases
saq_daily_warehouse.inventaire_cdm	cases	× uvc → bottles
saq_daily_warehouse.inventaire_cdq	cases	× uvc → bottles
saq_order_status.qty_commandee	cases	× uvc → bottles
saq_product.format_caisse	bottles/case	12 (spirits), 24 (RTD c24), 6 (RTD c6)

Warehouse Stock Fields Explained

Three columns from `saq_daily_warehouse` that are easy to confuse:

Column	Meaning
<code>inventaire_cdm</code>	Gross physical cases in the MTL warehouse — everything physically present
<code>inv_alloc_cdm</code>	MTL cases not yet allocated to any SAQ store order — "free stock"; can be 0 even when there's physical stock (meaning all cases are committed to outgoing deliveries but not yet shipped)
<code>inventaire_cdq</code>	Gross physical cases in QC warehouse (no allocation equivalent exists for QC)

For reorder calculations, use `inventaire_cdm` (gross). `inv_alloc_cdm` is shown as an informational column in the views.

Supabase Views

All views are in `scripts/create_dashboard_views.sql`. Deploy by running the file in Supabase SQL Editor.

Before redeploying: If renaming any column, run `DROP VIEW IF EXISTS view_name;` first. `CREATE OR REPLACE VIEW` does not allow column renames.

View 1: `v_po_prediction` — Reorder Alerts

Purpose: Shows weeks of stock remaining per product and flags products that need to be reordered.

Sales Velocity — Patrick's Anomaly-Cleaned Average:

- 1. Aggregate all weekly sales per product (all historical data)
- 2. Rank weeks highest → lowest; compute a **baseline average** from all weeks *except the top 4*
- 3. Define threshold = baseline × 1.5
- 4. **Remove ALL weeks above threshold** (not just the top 4 — catches all spike weeks)
- 5. Final average = mean of all remaining "normal" weeks
- 6. If product has fewer than 5 weeks of data, skip anomaly removal and use all weeks

Stock Calculation:

```
total_warehouse_bottles = (inventaire_cdm + inventaire_cdq) × uvc
weeks_of_stock = total_warehouse_bottles / avg_weekly_bottles
```

Alert Logic:

Alert	Condition
<code>COMMANDER</code>	<code>weeks_of_stock < 4</code>
<code>SURVEILLER</code>	<code>weeks_of_stock < 6</code>

Alert	Condition
OK	weeks_of_stock ≥ 6
Pas de ventes	No sales history

Key columns: warehouse_mtl_cases, warehouse_mtl_free (informational), warehouse_qc_cases, total_warehouse_bottles, avg_weekly_bottles, weeks_used, weeks_of_stock, alerte

View 2: v_yoy_sales — Year-over-Year Comparison

Purpose: Period-level sales comparison, current year vs. prior year.

Columns: code_saq, product_name, periode, bottles_this_year, bottles_last_year, bottles_diff, yoy_pct, revenue_this_year, revenue_last_year

View 3: v_open_orders — Open Purchase Orders Pipeline

Purpose: All open SAQ orders with date countdown.

Columns: All columns from saq_order_status + jours_avant_expedition (days until requested ship date, negative = overdue)

View 4: v_store_inventory_summary — Store Inventory by Product

Purpose: Latest store inventory snapshot, rolled up per product.

Columns: code_saq, product_name, nb_succursales, total_bouteilles, moy_par_succursale, succursales_rupture (stores with 0 bottles)

View 5: v_store_inventory_by_banner — Store Inventory by Banner

Purpose: Same as view 4 but split by banner type.

View 6: v_sales_trend — Weekly Sales Trend

Purpose: Weekly sales for all products, last 2 years.

Columns: code_saq, annee, periode, semaine, total_bottles, total_revenue, stores_sold

View 7: v_dashboard_kpi — Top-Level KPI Numbers

Purpose: Single-row summary for the top of the COO dashboard.

Returns: total_products, low_stock_products, open_orders, total_cases_on_order, active_stores, total_bottles_in_stores, total_warehouse_cases, last_warehouse_update, last_store_update

View 8: `v_sku_summary` — Single Source of Truth per SKU ☆

Purpose: The primary view for purchasing decisions. Combines warehouse stock + inbound orders + sales velocity + lead time into one row per product.

Stock columns:

- `warehouse_mtl_cases` — gross physical MTL cases (`inventaire_cdm`)
- `warehouse_mtl_free` — MTL cases not yet allocated (`inv_alloc_cdm`, informational)
- `warehouse_qc_cases` — gross physical QC cases
- `warehouse_bottles` — total warehouse in bottles
- `inbound_cases` / `inbound_bottles` — open orders with status "Attente" or "Attente récept." (not yet received)
- `effective_stock_bottles` — warehouse + inbound

Velocity: Same anomaly-cleaned average as `v_po_prediction`.

Weeks of stock:

- `weeks_of_stock` — warehouse only ÷ avg weekly demand
- `weeks_of_stock_total` — (warehouse + inbound) ÷ avg weekly demand

Alert: Based on `weeks_of_stock_total` (same thresholds as `v_po_prediction`)

Lead time: From `lead_times` table, stored as `lead_time_days`, shown as `lead_time_weeks` (÷ 7). Default 72 days (~10 weeks) when not set.

Sales Rep Cockpit View: `vw_cockpit_product_summary` ⚠ NOT YET DEPLOYED

File: `salesrep_cockpit/sql/vw_cockpit_product_summary.sql` **Purpose:** Replicates the main WordPress SAQ sales table for Cherry River sales reps in Retool.

One row per product (~80) with columns: `code_saq` | `product_name` | `p11ly` | `p11ty` | `var_p11` | `s1` | `s2` | `s3` | `s4` | `ptdly` | `ptdty` | `var_ptd` | `cdm_i` | `cdq_i` | `nb_succ_ly` | `nb_succ_ty` | `totalLY` | `totalTY`

All sales columns are in **cases** (bottles ÷ `format_caisse`).

Deploy: Run `salesrep_cockpit/sql/vw_cockpit_product_summary.sql` in Supabase SQL Editor.

Retool Dashboards

Exported JSON files are in `Retool Dashboards/`:

- `C00%20Cockpit (1).json` — Operations dashboard
- `SKU%20Summary%20Dashboard.json` — SKU-level summary view

To restore: Retool → Import App → select JSON file.

Product Filtering

76 Cherry River SAQ product codes are hardcoded in `scripts/saq_weekly_update.py` in the `CHERRY_RIVER_CODES` set. All CSV uploads filter to only these codes.

SAQ Calendar

- 13 periods per year, ~4 weeks each
- Current: 2025 / Period 12 / Week 4
- Column naming in pivot tables: `P[period][week]` — e.g., `P121` = Period 12 Week 1

What Is Complete

Component	Status
Weekly pipeline (scraper → unzip → upload)	✓ Complete, running every Monday
GitHub Actions automation	✓ Active
Supabase tables & schema	✓ Deployed
All 8 dashboard views	✓ Deployed in Supabase
COO Cockpit Retool dashboard	✓ Deployed at <code>cherryriver2.retool.com</code>
Sales Rep Cockpit SQL view	✓ Written, needs to be deployed
n8n daily automation	✓ Fixed and running
Lead times table	✓ Populated with real values
Supabase access	✓ Francis and Patrick both have access

What Is Incomplete / Pending

Component	Status	Notes
Sales Rep Cockpit Retool dashboard	⦿ Pending	View is ready; Retool table not built yet
Google Drive n8n credentials	🔄 Recurring	Must be refreshed every 6 days — Patrick manages this

Immediate Next Steps (Sales Rep Cockpit)

1. **Deploy view** — open Supabase SQL Editor, paste & run `salesrep_cockpit/sql/vw_cockpit_product_summary.sql`
2. **Build Retool table** — create Resource Query on `vw_cockpit_product_summary`, bind to Table component
3. **Format columns** — Var% red if negative, NB Succ as `91 -9 (76/15)` combined string
4. **Add filters** — product_type dropdown (RTD / Spirits) + product name search bar

Key Files to Read First

Order	File	Why
1	HANDOFF.md	This document — full project overview
2	CLAUDE.md	Architecture reference and key patterns
3	salesrep_cockpit/HANDOFF.md	Detailed context for the active sub-project
4	salesrep_cockpit/sql/vw_cockpit_product_summary.sql	View ready to deploy
5	scripts/create_dashboard_views.sql	All existing views with full SQL
6	.github/workflows/saq_weekly_update.yml	Automation workflow